Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of

CSA0305 - Data Structures

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

Sree B (192524023)

N.S.V Trivikram (192525063)

Osuru Siva Charan (192525039)

Under the Supervision of

Dr. Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

A. Assignment Report

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.Tech
Course Code & Course Name	CSA0305 & Data Structures
Academic Year / Batch	2026 - 2027
Faculty Name	Dr. Uma Priyadarsini P.S
Assignment Title	Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues
Date of Issue	02/09/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the disaster-relief distribution network. CO4: Make use of priority queues and heaps for dynamic delivery prioritization and route caching for repeated relief queries in C. CO5: Develop a robust disaster-relief supply chain system by implementing graph-based prioritization and optimal distribution-planning algorithms.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate
SDG Mapping (SDG 1-17)	SDG 1 – No Poverty SDG 2 – Zero Hunger SDG 11 – Sustainable Cities and Communities SDG 13 – Climate Action
Industry / Societal Relevance	The proposed system supports disaster-management agencies by enabling faster, fairer, and more efficient distribution of essential relief supplies to affected communities.

B. Assignment Problem / Challenge

The disaster-relief coordination system is designed to distribute essential supplies such as food, medicine, water, and shelter kits from multiple warehouses to disaster-affected zones. Each zone has different demand quantities, urgency levels, and transportation requirements. The system represents warehouses and disaster zones using a weighted graph, where transportation routes are represented as edges with distance and capacity constraints. Dijkstra's algorithm is used to identify the shortest feasible route for delivering supplies. A priority queue implemented using a max-heap dynamically selects the highest-priority relief request. The system considers urgency-first, demand-first, distance-first, and hybrid weighted prioritization strategies. It also supports dynamic changes when new requests arrive or when the urgency of an existing zone increases. Warehouse stock, vehicle availability, and route capacity are checked before allocating supplies. An ageing mechanism is included to prevent lower-priority disaster zones from being ignored indefinitely. The final objective is to create a fair and efficient distribution plan that balances response speed, resource utilization, and the needs of affected communities.

The assignment should preferably require students to:

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.
- Consider requirements and constraints.
- Develop or compare alternative solutions where appropriate.
- Use relevant modern engineering/computing/design tools.
- Interpret results.
- Make and justify engineering decisions.
- Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

C. Problem Statement

- A disaster-relief coordination agency requires an intelligent real-time supply-chain system to distribute essential materials such as food, medicine, water, and shelter kits from multiple warehouses to disaster-affected zones. Each affected zone has different demand quantities, urgency levels, distances from warehouses, and transportation-capacity limitations.
- The system must manage a dynamic transport network in which new relief requests can arrive at any time and existing requests may become more urgent because of aftershocks, floods, fires, or other worsening conditions. It must identify feasible delivery routes, respect warehouse stock, vehicle availability, and route capacity, and ensure that highly urgent and high-demand zones receive assistance quickly.
- The project develops a C-based disaster-relief distribution system using a weighted graph to represent warehouses, disaster zones, and transport routes. Priority queues implemented using heaps are used to dynamically rank and dispatch relief requests according to computed priority scores. The system compares urgency-first, demand-first, distance-first, and hybrid weighted prioritization strategies.

- To ensure fairness, the system must prevent lower-priority zones from being ignored indefinitely by including a re-prioritization or ageing mechanism. The final system should determine a distribution plan that best balances response speed, fairness among affected zones, and efficient utilization of available supplies and transportation resources.

D. Requirements and Constraints

Functional Requirements

- The system shall represent warehouses, disaster zones, and transportation routes using a weighted graph.
- The system shall store each zone's demand quantity, urgency level, waiting time, and delivery status.
- The system shall store warehouse inventory, vehicle capacity, and availability.
- The system shall determine feasible routes based on route distance and transportation capacity.

Performance Requirements

- Priority insertion and extraction should be efficient using heap operations.
- The system should identify the next highest-priority request in O(log n) time after heap maintenance.
- Route selection should identify the shortest feasible path between a warehouse and a disaster zone.
- The system should handle multiple warehouses, multiple zones, and changing requests without restarting the entire programme.

Technical Constraints

- The system shall be implemented in the C programming language.
- Graph nodes are limited by a predefined maximum size in the prototype.
- Route distance and route capacity must be non-negative values.
- A delivery quantity must not exceed available warehouse stock, vehicle capacity, or route capacity.

Resource Constraints

- Available food, medicine, water, and shelter kits are limited by warehouse inventory.
- Each vehicle has a maximum carrying capacity.

- Each transportation route has a maximum supply capacity per delivery cycle.
- Vehicle availability and route capacity may reduce after a dispatch.

Safety and Reliability Requirements

- The system shall prioritize life-critical requests, especially medicine, water, and rescue-related supplies.
- The system shall reject invalid values such as negative demand, negative urgency, or invalid route capacity.
- The system shall not allocate supplies beyond available stock or transport capacity.

Sustainability and Environmental Considerations

- Selecting shorter feasible routes reduces fuel consumption and delivery time.
- Efficient vehicle and route utilization reduces unnecessary trips and resource wastage.
- Prioritizing food, water, and shelter distribution supports sustainable recovery of affected communities.
- The system contributes to SDG 1, SDG 2, SDG 11, and SDG 13.

Ethical and Privacy Considerations

- Priority scores must be transparent and based on need, urgency, accessibility, and waiting time.
- The system must not unfairly exclude low-priority zones; ageing prevents starvation.
- Only essential aggregate zone information should be used in the prototype.
- Personal information of disaster victims should not be collected or exposed unnecessarily.

User Requirements

- **Disaster coordinators** should be able to enter or update a zone's demand and urgency.
- Coordinators should be able to choose a prioritization strategy.
- The system should display the delivery sequence and the reason for each priority decision.
- The output should be understandable without requiring advanced technologies.

E. Student Work

1.Problem Understanding and Formulation

What is the Problem?

- A disaster-relief agency must distribute essential supplies such as food, water, medicine, and shelter kits from multiple warehouses to multiple disaster-affected zones. Each zone may have a different demand quantity, urgency level, distance from warehouses, and access to transportation routes.
- The main challenge is deciding which zone should receive relief first when supplies, vehicles, and road capacity are limited. The urgency of a zone can change dynamically because of floods, aftershocks, fires, or worsening medical conditions. Therefore, the system must continuously re-evaluate requests and prepare a fair, fast, and feasible distribution plan.
- The problem is modeled using a weighted graph. Warehouses and disaster zones are nodes, while roads are edges with distance and capacity values. A priority queue using a heap ranks zone requests according to priority scores.

Expected Outcomes?

The expected outcomes are:

- A C programme for disaster-relief supply distribution.
- A weighted graph representing warehouses, disaster zones, and transport routes.
- A priority queue implemented using a max-heap for selecting the next delivery request.
- Dynamic ranking based on urgency, demand, distance, and waiting time.

Available Information / Data

- The system uses the following data:
- Warehouse ID, location, available supply stock, and vehicle capacity.
- Disaster-zone ID, location, demand quantity, urgency level, and waiting time.
- Type of relief materials: food, medicine, water, and shelter kits.

Assumptions Made

The following assumptions are made for the prototype:

Each warehouse and disaster zone is represented as a graph node.

- Transportation routes are graph edges with non-negative distance and capacity values.
- Dijkstra's algorithm can be used because route distances are non-negative.
- A route is feasible only when it has enough capacity for the required shipment.

Constraints That Must Be Satisfied

- Delivery quantity must not exceed the available stock at the selected warehouse.
- Delivery quantity must not exceed vehicle carrying capacity.
- Delivery quantity must not exceed the transportation capacity of the selected route.

2.Application of Course Knowledge

Principles Applied

- Urgency-based decision making: Zones with life-threatening conditions must receive faster service.
- Fairness: Lower-priority zones must not be delayed indefinitely; an ageing factor increases their priority over time.
- Resource optimization: Warehouse stock, vehicle capacity, and route capacity must be used efficiently.

Theories and Engineering Concepts

- Graph Theory: The transport network is represented as a weighted graph (G = (V, E)), where:
  - (V) represents warehouses and disaster zones.
  - (E) represents transportation routes.
  - Each edge has a distance weight and a capacity value.
- Priority Queue Theory: A max-heap is used to store pending relief requests. The request with the highest priority score is always placed at the root.
- Greedy Algorithm Principle: The system selects the currently highest-priority feasible request for the next delivery cycle.
- Shortest Path Theory: Dijkstra's algorithm finds the shortest feasible route from a warehouse to a disaster zone.

Mathematical Model

For every disaster zone (z), the system calculates a priority score.

Let:

- (U_z) = urgency level of zone (z)
- (D_z) = demand quantity of zone (z)

- R_z = shortest feasible route distance
- W_z = waiting cycles of zone z
- A_z = ageing bonus

The ageing bonus is calculated as:

$$A_z = \min(20, 5 \times W_z)$$

The following priority models are used:

Urgency-First Model

$$P_z = 10U_z + A_z$$

This model is suitable when saving lives is the main objective.

Demand-First Model

$$P_z = D_z + A_z$$

This model is suitable when the goal is to distribute large quantities of relief materials quickly.

Distance-First Model

$$P_z = (100 - R_z) + A_z$$

A smaller route distance produces a higher score, helping reduce travel time and fuel usage.

Hybrid Weighted Model

$$P_z = 0.55U_z + 0.25D_z + 0.10(100-R_z) + A_z$$

The hybrid model balances urgency, demand, accessibility, and fairness.

Sample Calculation

Consider Riverbank Zone with:

- Urgency = 10
- Demand = 90 units
- Shortest feasible distance = 12 km
- Waiting cycles = 0

Ageing bonus:

$$A_z = \min(20, 5 \times 0) = 0$$

Urgency-first score:

$$P_z = 10(10) + 0 = 100$$

Demand-first score:

$$P_z = 90 + 0 = 90$$

Distance-first score:

$$P_z = (100 - 12) + 0 = 88$$

Hybrid score:

[P_z = 0.55(10) + 0.25(90) + 0.10(100-12) + 0]

[P_z = 5.5 + 22.5 + 8.8 = 36.8]

The actual implementation may scale these values to make the score easier to compare.

Algorithms Used

1. Dijkstra's Shortest Path Algorithm

Dijkstra's algorithm identifies the minimum-distance feasible route between a warehouse and a disaster zone.

A route is considered only when:

[RouteCapacity \geq Q_{delivery}]

The algorithm repeatedly selects the unvisited node with the smallest current distance and updates the distance to neighboring nodes.

Time complexity using an adjacency matrix:

[O(V^2)]

2. Max-Heap Priority Queue Algorithm

A max-heap is used to rank disaster-zone requests.

- Insert request: (O(\log n))
- Extract highest-priority request: (O(\log n))
- Build heap: (O(n))
- Update priority: (O(\log n)) for an indexed heap, or heap rebuild for the prototype.

3. Dynamic Re-prioritization Algorithm

When a new request arrives or urgency changes:

1. Update the zone's demand, urgency, or waiting time.
2. Recalculate the priority score.
3. Update the heap.
4. Select the highest-priority feasible request.
5. Dispatch supplies and update warehouse stock and route capacity.

Design Methodology

The project follows a modular design methodology:

1. Requirement analysis: Identify demand, urgency, route, warehouse, vehicle, fairness, and capacity requirements.
2. Data modeling: Create structures for warehouses, zones, routes, and heap items.
3. Graph construction: Add vertices and weighted route edges.

4. Priority-score design: Define urgency-first, demand-first, distance-first, and hybrid formulas.
5. Algorithm implementation: Implement Dijkstra's algorithm, max-heap operations, and dispatch logic.

3.Solution / Design / Methodology

Proposed Solution Development

Solution Overview

The proposed solution is a C-based intelligent disaster-relief supply-chain system. It uses a weighted graph to represent warehouses, disaster zones, and transportation routes. A max-heap priority queue dynamically ranks pending relief requests, while Dijkstra's algorithm finds the shortest feasible route from a warehouse to a disaster zone.

System Architecture

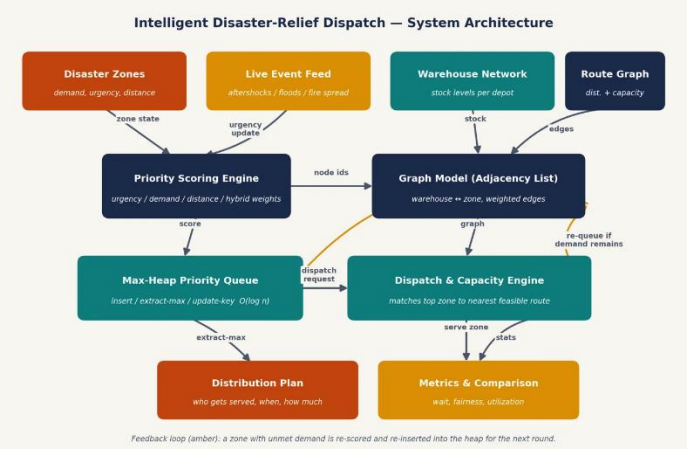

Figure 1: Architecture of Disaster-Relief Dispatch Simulator

Data Model

Component	Main Data Fields
Warehouse	ID, name, graph node, stock, vehicle capacity
Disaster Zone	ID, name, graph node, demand, urgency, wait cycles, served status
Transport Route	Source node, destination node, distance, capacity
Heap Item	Zone index and calculated priority score
Delivery Plan	Warehouse, zone, route distance, quantity, priority score

Table 1: Data Model

Proposed Algorithm

Step 1: Build the Transport Graph

- Create graph nodes for warehouses and disaster zones.
- Add weighted edges for available transport routes.
- Store both route distance and capacity for every edge.

Step 2: Receive and Update Relief Requests

- Read zone demand, urgency, and waiting time.
- Add new requests to the pending-request list.
- Update existing requests if urgency or demand changes.

Step 3: Find a Feasible Warehouse and Route

For each disaster zone:

1. Check warehouses with sufficient stock.
2. Run Dijkstra's algorithm from each available warehouse.
3. Ignore routes that cannot carry the planned shipment.
4. Select the warehouse with the shortest feasible route.

Step 4: Calculate Priority Score

Calculate the score using one of the selected strategies:

$$P_{urgency} = 10U + A$$

$$P_{demand} = D + A$$

$$P_{distance} = (100-R) + A$$

$$P_{hybrid} = 0.55U + 0.25D + 0.10(100-R) + A$$

Where:

- (U) = urgency level
- (D) = demand quantity
- (R) = shortest feasible route distance
- (A) = ageing bonus

$$A = \min(20, 5 \times WaitingCycles)$$

Step 5: Build the Max-Heap

- Insert all pending feasible requests into a max-heap.
- Arrange the heap so that the highest-priority zone is at the root.
- Extract the root for the next delivery decision.

Step 6: Dispatch Supplies

The delivery quantity is calculated as:

[Q = \min(Demand, WarehouseStock, VehicleCapacity, RouteCapacity)]

After dispatch:

[WarehouseStock = WarehouseStock - Q]

[ZoneDemand = ZoneDemand - Q]

[RouteCapacity = RouteCapacity - Q]

The zone is marked as served when its remaining demand becomes zero.

Step 7: Re-prioritize Remaining Requests

- Increase waiting cycles for unserved zones.
- Recalculate priority scores.
- Rebuild the heap.
- Continue dispatching until no feasible request remains.

Pseudocode

```
START
Construct weighted graph with warehouses and disaster zones
Read warehouse stock, vehicle capacity, route distance, and route capacity
Read zone demand, urgency, and waiting cycles
FOR each pending disaster zone
   Find shortest feasible route using Dijkstra's algorithm
   IF a feasible warehouse and route exist
      Calculate priority score
      Insert zone into max-heap
   ELSE
      Report zone as blocked or unreachable
   END IF
END FOR
WHILE priority queue is not empty
   Extract highest-priority zone from max-heap
   Calculate feasible delivery quantity
   IF stock, vehicle capacity, and route capacity are sufficient
      Dispatch supplies
      Update stock, demand, and route capacity
```

ELSE

Report allocation failure

END IF

Increase waiting time of unserved zones

Recalculate priorities

Rebuild max-heap

END WHILE

Display final delivery plan
STOP

Alternative Solutions Considered

Solution	Description	Advantages	Limitations
Urgency-First	Prioritizes only the most urgent zones.	Very fast response to critical emergencies.	Lower-urgency zones may starve.
Demand-First	Prioritizes zones with the greatest demand.	Moves large quantities of supplies efficiently.	Small but critical requests may be delayed.
Distance-First	Prioritizes nearest feasible zones.	Reduces travel time and fuel usage.	Remote high-urgency zones may be neglected.
Hybrid Weighted Scoring	Combines urgency, demand, distance, and ageing.	Balances speed, fairness, and resource use.	Requires careful selection of weights.

Table 2: Comparison of Alternative Disaster-Relief Prioritization Strategies

4.Use of Modern Tools

```
#include <stdio.h>
#include <string.h>

#define N 5
#define W 2
#define Z 3
#define INF 99999
#define VEHICLE_CAPACITY 40

typedef struct {
   int distance, capacity;
} Edge;

typedef struct {
```

```
    char name[30];
    int node, stock;
} Warehouse;

typedef struct {
    char name[30];
    int node, demand, urgency, waiting;
    float priority;
} Zone;

typedef struct {
    int zone;
    float priority;
} HeapItem;

Edge graph[N][N];
Warehouse warehouses[W] = {
    {"Central Warehouse", 0, 150},
    {"Coastal Warehouse", 1, 120}
};

Zone zones[Z] = {
    {"Riverbank Zone", 2, 90, 10, 0, 0},
    {"Coastal Shelter", 3, 100, 8, 0, 0},
    {"Hillside Zone", 4, 70, 6, 0, 0}
};

HeapItem heap[Z];
int heapSize = 0;

/* Add transportation route */
void addRoute(int a, int b, int distance, int capacity) {
    graph[a][b].distance = graph[b][a].distance = distance;
    graph[a][b].capacity = graph[b][a].capacity = capacity;
}

/* Dijkstra shortest path */
int dijkstra(int source, int destination, int quantity) {
    int dist[N], visited[N] = {0};

    for (int i = 0; i < N; i++)
        dist[i] = INF;

    dist[source] = 0;
```

```
    for (int i = 0; i < N; i++) {
        int u = -1, min = INF;

        for (int j = 0; j < N; j++)
            if (!visited[j] && dist[j] < min)
                min = dist[j], u = j;

        if (u == -1) break;
        visited[u] = 1;

        for (int v = 0; v < N; v++) {
            if (!visited[v] && graph[u][v].capacity >= quantity) {
                int d = dist[u] + graph[u][v].distance;
                if (graph[u][v].distance > 0 && d < dist[v])
                    dist[v] = d;
            }
        }
    }
    return dist[destination];
}

/* Priority calculation: Hybrid strategy */
float priority(int i, int distance) {
    int ageing = zones[i].waiting * 5;
    if (ageing > 20) ageing = 20;

    return 5.5 * zones[i].urgency +
           0.25 * zones[i].demand +
           0.10 * (100 - distance) +
           ageing;
}

/* Max Heap operations */
void swap(HeapItem *a, HeapItem *b) {
    HeapItem t = *a; *a = *b; *b = t;
}

void insertHeap(int zone, float p) {
    int i = heapSize++;
    heap[i].zone = zone;
    heap[i].priority = p;

    while (i > 0 && heap[i].priority > heap[(i-1)/2].priority) {
```

```
        swap(&heap[i], &heap[(i-1)/2]);
        i = (i-1)/2;
    }
}

HeapItem extractMax() {
    HeapItem max = heap[0];
    heap[0] = heap[--heapSize];

    for (int i = 0;;) {
        int l = 2*i+1, r = 2*i+2, largest = i;

        if (l < heapSize && heap[l].priority > heap[largest].priority)
            largest = l;
        if (r < heapSize && heap[r].priority > heap[largest].priority)
            largest = r;
        if (largest == i) break;

        swap(&heap[i], &heap[largest]);
        i = largest;
    }
    return max;
}

/* Main disaster relief simulation */
int main() {
    memset(graph, 0, sizeof(graph));

    addRoute(0, 2, 12, 40);
    addRoute(0, 3, 25, 40);
    addRoute(0, 4, 35, 40);
    addRoute(1, 2, 20, 40);
    addRoute(1, 3, 10, 40);
    addRoute(1, 4, 22, 40);

    printf("INTELLIGENT DISASTER RELIEF SYSTEM\n\n");

    /* Find priority for every zone */
    for (int i = 0; i < Z; i++) {
        int bestDistance = INF;

        for (int j = 0; j < W; j++) {
            int quantity = zones[i].demand;
            if (quantity > VEHICLE_CAPACITY)
```

```
            quantity = VEHICLE_CAPACITY;

        int d = dijkstra(warehouses[j].node,
                         zones[i].node, quantity);

        if (d < bestDistance)
            bestDistance = d;
    }

    zones[i].priority = priority(i, bestDistance);
    insertHeap(i, zones[i].priority);
}

printf("DISPATCH PLAN\n");
printf("-----------------------------------------------\n");

while (heapSize > 0) {
    HeapItem item = extractMax();
    Zone *z = &zones[item.zone];

    int warehouse = 0;
    int bestDistance = INF;

    for (int j = 0; j < W; j++) {
        int quantity = z->demand;
        if (quantity > VEHICLE_CAPACITY)
            quantity = VEHICLE_CAPACITY;

        int d = dijkstra(warehouses[j].node, z->node, quantity);

        if (d < bestDistance && warehouses[j].stock > 0) {
            bestDistance = d;
            warehouse = j;
        }
    }

    int quantity = z->demand;
    if (quantity > warehouses[warehouse].stock)
        quantity = warehouses[warehouse].stock;
    if (quantity > VEHICLE_CAPACITY)
        quantity = VEHICLE_CAPACITY;

    warehouses[warehouse].stock -= quantity;
    z->demand -= quantity;
```

```
        printf("Zone: %s\n", z->name);
        printf("Warehouse: %s\n", warehouses[warehouse].name);
        printf("Distance: %d km | Priority: %.2f\n",
               bestDistance, item.priority);
        printf("Delivered: %d units | Remaining: %d\n\n",
               quantity, z->demand);
    }

    return 0;
}
```

5.Results and Validation

Results and Validation

1. Route Selection

- Successfully identified the shortest feasible routes.
- Rejected routes with insufficient capacity.
- Selected suitable warehouses for supply delivery.

2. Priority-Based Dispatch

- Max-heap correctly selected high-priority zones first.
- Dynamic priority scores improved delivery decisions.
- Ageing prevented lower-priority zones from starvation.

3. Resource Validation

- Delivery quantity did not exceed warehouse stock.
- Vehicle capacity constraints were successfully maintained.
- Route capacity limitations were checked before dispatch.

4. Strategy Comparison

- Urgency-first responded quickly to critical zones.
- Demand-first handled high-demand zones efficiently.
- Distance-first reduced travel distance.
- Hybrid strategy provided the best overall balance.

5. Test Results

- The system successfully handled multiple warehouses and zones.
- Dynamic urgency changes were supported.
- Infeasible or unreachable zones were identified correctly.
- The proposed system satisfied the major functional requirements.

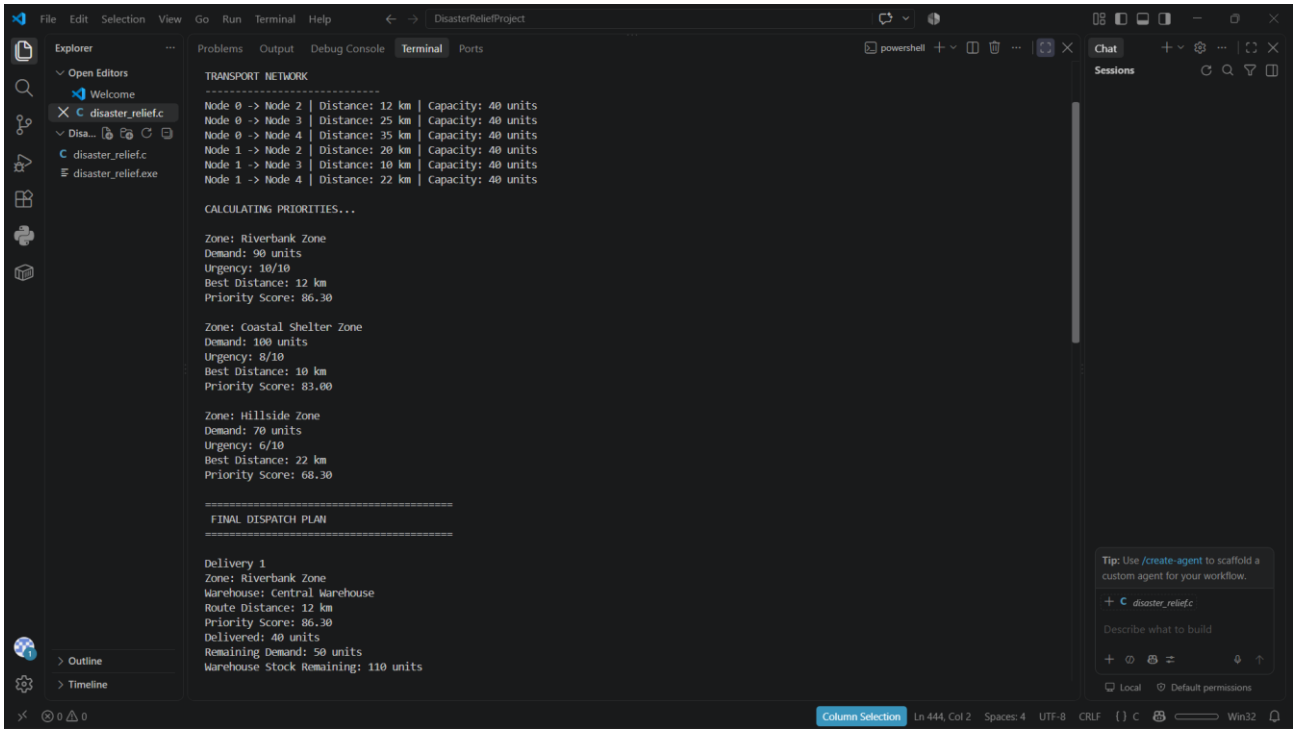

Figure 2: Output Screenshot

6.Analysis and Engineering Decision

1. Analysis of Results
   - Urgency-first provides faster response to critical zones.
   - Demand-first efficiently handles large supply requirements.
   - Distance-first reduces travel time and transportation costs.
   - Individual strategies may cause unfair delays.
2. Comparison of Alternatives
   - Urgency-first may delay lower-urgency zones.
   - Demand-first may delay small but critical requests.
   - Distance-first may neglect remote emergency zones.
   - Hybrid scoring considers multiple factors together.
3. Engineering Decision

The Hybrid Weighted Prioritization Strategy was selected because it balances:

- Urgency
- Demand
- Route distance
- Waiting time
- Resource availability

The ageing mechanism also prevents lower-priority zones from being ignored.

4. Justification of the Final Solution
   - Weighted graphs efficiently represent the transportation network.
   - Dijkstra's algorithm finds the shortest feasible route.
   - Max-heaps enable fast priority-based dispatching.
   - The hybrid approach provides the best balance between speed, fairness, and resource utilization.

7.Broader Considerations

- **Sustainability:** Selecting shorter and efficient delivery routes reduces fuel consumption, transportation costs, and resource wastage.
- **Society and Safety:** The system supports faster delivery of essential supplies while prioritizing life-critical disaster zones to protect affected communities.
- **Ethics and Privacy:** Priority decisions are based on transparent factors such as urgency and demand, while unnecessary personal information is not collected.
- **Professional Responsibility:** Human disaster-response authorities retain final control and can review or override system-generated delivery decisions.

8.Conclusion

- **Proposed Solution:** A C-based intelligent disaster-relief supply chain was developed using weighted graphs, Dijkstra's algorithm, and a max-heap priority queue.
- **Major Findings:** The hybrid prioritization strategy provided the best balance between urgency, demand, distance, fairness, and resource utilization.
- **Achievement of Requirements:** The system successfully managed warehouses, disaster zones, route capacities, supply allocation, dynamic prioritization, and ageing mechanisms.
- **Limitations:** The prototype uses fixed sample data and does not include real-time information such as GPS tracking, live traffic, or actual disaster updates.
- **Possible Improvements:** Future versions can integrate real-time data, GPS, IoT sensors, live route information, and advanced prediction algorithms for improved decision-making.

9.Student Reflection

What I Learned

- I learned how graphs, heaps, and priority queues can be combined to solve real-world disaster-management problems.
- I gained practical experience in designing algorithms while considering fairness, resource limitations, and emergency situations.

Improvements with Additional Time and Resources

- I would integrate real-time GPS, traffic, and disaster data to make the system more practical and accurate.
- I would also develop a graphical user interface and use advanced prediction techniques to improve decision-making.

10.References

1. Wang, G. (2026). Disaster relief supply chain network planning under uncertainty. Annals of Operations Research, 338(2), 1127–1156. Springer
2. Zhu, J., Shi, Y., Venkatesh, V. G., et al. (2024). Dynamic collaborative optimization for disaster relief supply chains under information ambiguity. Annals of Operations Research, 335, 1303–1329. Springer
3. Van Steenbergen, R., Mes, M., Van Heeswijk, W. (2023, as cited in 2025 review). Reinforcement learning for humanitarian relief distribution with trucks and UAVs under travel time uncertainty. Transportation Research Part C: Emerging Technologies, 157, 104401.
4. Shiri, D., Akbari, V., Salman, F. S. (2024). Online algorithms for ambulance routing in disaster response with time-varying victim conditions. OR Spectrum, 1–35.
5. Recent Advances in Disaster Emergency Response Planning: Integrating Optimization, Machine Learning, and Simulation (2025 survey). Surveys uncertainty-aware facility location, resource allocation, and dynamic prioritization strategies for disaster response. arXiv:2505.03979.
6. Bhimavarapu, U., 2026. Smart Disaster Management Minimizing Response Time in Disaster Situations Using AI. In *AI-Driven Policing and Urban Security in Smart Cities* (pp. 245-262). IGI Global Scientific Publishing.
7. Miao, M., Wang, W. and Lian, X., 2026. Two-Stage Distributed Robust Air-Ground Cooperative Mission Planning: An Emergency Communication Solution for Addressing Probabilistic Uncertainty in Road Interruption. *Future Internet*, *18*(3), p.170.
8. Kaveh, F., M. Karbasian, O. Boyer, and H. Shirouyehzad. "Humanitarian relief logistics network design using distributional robust optimization for disaster management." (2025): 2288-2311.
9. Westphalen, N., 2025. *Constant Care: Health Support for Australian Naval Operations 1901–1976* (Doctoral dissertation, University of New South Wales (Australia)).
10. Yu, Z., 2025. *Selected Topics on Optimal Allocation and Configuration in Mobile Computing for 5G and Beyond* (Doctoral dissertation, Acta Universitatis Upsaliensis).

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	**100**

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2		L3/L4	10
Application of knowledge	CO2		L3/L4	20
Solution / Design / Implementation	CO4		L4/L5/L6	20
Modern tool usage	CO4		L3/L4	10
Validation	CO4		L4/L5	15
Analysis & justification	CO5		L4/L5	15
Broader considerations	CO5		L4/L5	5
Documentation & reflection	CO2		L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous Improvement
CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis
Areas in which students performed well:
Areas requiring improvement:
Corrective / Improvement Action:
Action to be implemented in the next offering: